

Demo Video — Recording Script

Multi-Agent Business Analyst · target length 7:00 · silent screen capture + voiceover

Every number quoted below is real and comes from `runs/run-20260808-202457.json` or from re-running the retriever, so nothing here will contradict what is on screen.

Pre-flight checklist

Do all of this *before* you hit record. Most demo videos are ruined by one of these, not by the content.

Window	Chrome at 1920×1080, bookmarks bar hidden (<code>Ctrl+Shift+B</code>), one tab only
Zoom	Browser at 80% (<code>Ctrl+-</code> twice) — the six audit tabs and the sidebar both fit without scrolling
Notifications	Windows Focus Assist on. A Teams toast mid-take costs you the whole run
Terminal	Font size up to ~16pt. Default terminal text is unreadable at 1080p after compression
The cache	Leave <code>.cache/11m/</code> in place. "Why did profit decline?" is fully cached, so a re-run is free — see the note under Shot 4
Dry run	Walk the whole thing once without recording. You will find one thing that has moved

Recorder: OBS Studio (free) — Display Capture, 1080p30, MP4. Record video first with no talking, then lay the voiceover over it in the OBS/Clipchamp timeline. Recording both at once means one stumble kills a good screen take.

Slides needed: only two, both in Shot 2 and Shot 3. You can screenshot the ASCII diagrams straight out of `README.md` §3 and §4 rather than rebuilding them in PowerPoint — they are already clear, and they prove the diagram lives in the repo.

Shot 1 — Cold open

[0:00 – 0:22]

Screen: The finished report, already on screen — scrolled to the `CONDITIONAL GO` block with the four metric tiles visible above it (Decision, Confidence 68%, Overall risk ★★★★★, Judge score 82/100).

Do: Nothing. Hold the frame completely still.

Say:

This is a question a business analyst gets asked: *why did profit decline?* And this is the answer a team of nine AI agents produced — not a summary of a document, but a decision, with a recommendation, a confidence number, and a list of the things it could not verify.

What I want to show you is not the answer. It's how it got there, and how I know it isn't making the numbers up.

Note: open on the *output*, not on a title card. You have about eight seconds before an examiner decides whether this is interesting.

Shot 2 — The problem

[0:22 – 1:15]

Screen: Slide — the architecture diagram from `README.md` §3.

Do: Reveal or highlight the diagram top-to-bottom as you speak — planner, then the five specialists, then risk, then strategy.

Say:

The hard part of business analysis isn't finding data. The analyst already has the financials, the CRM export, the support tickets, the competitor tracker. The hard part is synthesis — because no single document contains the answer.

In this company, the margin decline in the income statement is really a vendor problem in the operations file, and it shows up a third time as a complaint theme in the support tickets. One model reading one file will never see that.

So instead there's a planner that breaks the question into assignments; five domain specialists — finance, sales, customer, market, operations — that work concurrently on their own evidence; a risk agent that reads all of their findings at once; and a strategy agent that has to actually commit to an answer. Then a ninth agent grades the result.

Shot 3 — The data room

[1:15 – 1:50]

Screen: The app's  **Data room** tab, showing the inventory report.

Do: Scroll slowly through the file inventory once. Don't stop to read individual rows.

Say:

Everything the team can see is here — seventeen files describing Nimbus Audio: finance, sales, customer, market, operations. Nothing else is available to it.

The company is synthetic, and that's deliberate. Real company data can't be published with an academic submission — but more usefully, a generated data room lets me plant the answer in advance. Revenue grows while margin and cash quietly deteriorate. So whether the agents find it is a test, rather than an anecdote.

Shot 4 — The live run

[1:50 – 3:20]

Screen: The app's main page. Type the question, press **Run the analysis**, and let the status box stream.

Do: 1. Click the "Why did profit decline?" example button — it fills the box for you. 2. Press **Run the analysis**. 3. Hold on the streaming trace. Don't touch anything.

⚠ Read this before recording Shot 4

With the cache on, this run finishes in **seconds**, because all nine calls are already on disk. That's great for a live demo and terrible for a video — the trace you want to talk over will be gone before your first sentence.

Pick one:

Option	What to do	Cost
A — recommended	Record the fast cached run, then <i>slow the clip down</i> to about 25% in your editor for this shot. The timestamps in the trace stay legible and honest	free
B	Untick "Reuse cached responses" in the sidebar and record a genuinely live run, then cut the middle out	~\$0.26, ~3.5 min
C	Record the cached run at full speed and simply say "this is a re-run, served from cache — the first run took three and a half minutes"	free

Option A gives the best video. Option B is the most honest if you're asked live. Either way, **say which one you did** — Shot 8 covers the cost numbers anyway.

Say:

The planner goes first. It restates the decision, breaks it into six sub-questions, and writes one assignment per specialist — telling finance to build the bridge from prior profit to current profit, telling operations to test whether COGS inflation or vendor pricing explains it.

Then the five specialists go at once. They share no state, so there's nothing to serialise — each one took about fourteen seconds, overlapping.

Risk can't start until all five are in, because its whole job is to find the compound risks that no single specialist can see. That one took fifty-six seconds. Strategy waits on risk, and takes another forty-eight. Those two are the expensive part of the run, and that's exactly why — they reason over every finding at once.

Nine model calls. Two hundred and three seconds.

Shot 5 — The report

[3:20 – 4:20]

Screen:  **Report** tab. Start at the four metric tiles, scroll into the executive summary.

Do: Scroll slowly. Pause on the confidence rationale line.

Say:

Conditional go. Launch a CFO-led operating-margin recovery program within thirty days — but hold further discount-led volume expansion until the contribution bridge is reconciled.

Three things in that output are the entire point of the design.

The numbers are real and traceable: revenue up nineteen point one percent to fifty-six point four million, but gross margin down one point two points, and net income down forty-six percent.

The conclusion crosses four domains — logistics cost, manufacturing variance, sales discounting, and working capital — and no single file in that data room contains it.

And the confidence is sixty-eight percent, argued *down* from certainty with named gaps: sales revenue doesn't reconcile to finance's period, and product margins aren't available. It didn't assert a number; it justified one.

Shot 6 — Where the numbers come from

[4:20 – 5:25]

Screen: Split this shot in two. - First half: `analyst/analytics.py` open in the editor, scrolled through quickly. - Second half: the app's  **Evidence** tab, with `competitor pricing pressure` typed into the retriever box.

Do: In the Evidence tab, type the query and let the ranked passages render. Point at the `dense #2` / `dense #3` annotations under the fused scores.

Say:

This is the design decision the project turns on.

Language models can't be trusted to do arithmetic over a CSV, so they're never asked to. Every figure an agent sees — growth rates, margin bridges, concentration ratios, days of supply — is computed in Python first, in this one file. The agent's job is interpretation, not calculation.

The prose half is handled differently. Twenty-three passages, indexed two ways: BM25 for exact terms, and a latent-semantic index for meaning. Watch what happens on "competitor pricing pressure" — BM25 gets the top hit, but passages two through five are found *only* by the dense retriever, because they discuss cell price increases and a competitor's launch price without ever using the word "pressure".

The two are combined by reciprocal rank fusion rather than by adding scores, because BM25 scores are unbounded and cosine similarities live between minus one and one. Ranks are the only scale the two share.

Shot 7 — The judge

[5:25 – 6:25]

Screen:  **Evaluation** tab. Scroll to the unsupported-claims list.

Do: Hold on the two [high] findings long enough to read them. This is the strongest sixty seconds in the video — don't rush it.

Say:

A report that reads well and cites a number that doesn't exist is the characteristic failure of this kind of system, and it's invisible to anyone who hasn't read the source data.

So a ninth agent grades the finished report against the exact evidence the specialists were shown. Eighty-two out of a hundred. Verdict: revise.

And it's not being pedantic. It rejected the report's central framing — the report dismissed a revenue-volume problem, and the judge pointed out the latest six months show a five point seven percent revenue decline. Then it caught something subtler: the report treated a variance against budget as if it were a deterioration over time. A variance to budget may just mean the budget was wrong.

Note what it does *not* flag — figures the report derived by arithmetic. An earlier version of this prompt marked every computed growth rate as unsupported because the number didn't appear verbatim, and that made the judge useless. It cried wolf on every calculation. Fixing that is the difference between a judge worth reading and a noise generator.

Shot 8 — Reliability and cost

[6:25 – 7:00]

Screen: Terminal. Run the test suite live:

```
python -m unittest discover -s tests -t .
```

Then cut to the ↓ **Export** tab showing the usage JSON.

Do: Let the tests actually run to green on camera — it takes about four seconds, which is a perfect beat.

Say:

Forty-seven tests, four seconds, no API key and no network — everything runs on a mock backend. And they're built around defects that actually happened, not around coverage. The call ceiling was read-then-decide, so five specialists starting together all saw the same under-budget count, and a ceiling of two admitted six calls. That test races ten threads for three slots.

The run you just watched: nine calls, about eighty-seven thousand tokens, twenty-six cents, three and a half minutes. Re-asking the same question is free — it's served from the disk cache.

Shot 9 — Close

[7:00 – 7:30]

Screen: Back to the report, or a plain slide with the three limitations.

Do: Static frame. Let it breathe.

Say:

Three honest limitations. The pipeline is real but the company is not, so nothing here validates the agents against messy real-world data. The dense retriever is LSA fitted on this corpus, not a pretrained encoder — on a larger corpus that's the first component I'd replace, and it's a one-class swap. And the judge is a model too: it catches unsupported figures reliably, but it's weaker at catching an argument that's well-grounded and still wrong. It reduces the failure rate; it doesn't eliminate it.

Which is the point. Every part of this system is built to make its own reasoning auditable — the numbers computed in code, the evidence cited, the conclusion graded. Thank you.

The 3-minute cut

If you're given a hard three-minute limit, keep **1, 4, 5, 7** and nothing else: the answer, the run, the report, the judge. That sequence still makes the whole argument. Shots **2, 3, 6** and **8** are the ones to drop — they're supporting evidence, and a viva panel will ask about them anyway.

Things not to do

- **Don't read the report aloud on screen.** The viewer can read faster than you can speak. Talk about what it *means* while they read it.

- **Don't demo the sidebar controls.** Backend, effort, alpha, top-k — it's tempting because it looks configurable, but it's the least interesting sixty seconds available to you. Mention the mock backend in Shot 8 and move on.
- **Don't apologise for the synthetic data.** Shot 3 already frames it as a deliberate methodological choice, which is what it is. Raising it a second time turns a strength into a doubt.
- **Don't speed up the judge shot.** It's the part that distinguishes this from a wrapper around a chat model.